Metaheuristic Algorithms and Applications: Program Report III

Group 17 N26140804 張暐俊

1. Introduction

This assignment applies a metaheuristic algorithm to the Traveling Salesman Problem. The spec only requires choosing at least two algorithms, so I continued with the choices from the previous two programs and kept using **Genetic Algorithm (GA)** and **Particle Swarm Optimization (PSO)**. For one thing, I have now implemented these two algorithms twice in a row and am fairly familiar with them. For another, reusing the same pair of algorithms lets me focus on the question of how the same GA and PSO behave once moved to a combinatorial optimization problem, rather than getting to know yet another new method from scratch.

The dataset follows the spec's requirement, using the designated TSPLIB Kaggle dataset, specifically the entry with `instance_id == 24`. There was a small confusion when I first read the spec. The About Dataset paragraph states that the whole dataset is "2,783 instances, 20 cities each," yet the table further down the same page clearly marks instance 24 as having 52 cities, so the two numbers do not match. I ultimately took "instance 24 = 52 cities" as authoritative, because this is the very entry the spec actually asks me to handle, and the coordinates that actually load in are indeed 52 of them. Every experiment that follows is built on these 52 cities.

Then, also following the spec, the experiment is split into two parts. *Part I* runs GA and PSO on three subsets of the first 12, 30, and 52 cities, comparing them with the four metrics the spec requires. *Part II* takes only the first 30 cities and additionally imposes the constraint that even-numbered cities cannot be connected to each other, so its fitness differs from *Part I*.

Finally, the Discussion likewise responds to the spec's two sub-questions, documenting all parameter settings and plotting the best routes of both Parts.

2. Program design (Discussion (i))

Here, besides presenting some of the design at the code level, I also first address the Discussion (i) required by the spec, covering each algorithm's solution encoding, fitness, selection, crossover, mutation, and stopping condition. As for the actual numerical parameter values, to avoid losing focus I have collected them into the comparison table under the later Parameter Settings section.

2.1 Data Pipeline

The raw CSV is about 226 MB, where one row is one TSP instance. My approach is to read it in and immediately filter out the single row with `instance_id == 24`, then use `ast.literal_eval` to restore the two string columns `city_coordinates` and `distance_matrix` back into numeric arrays. Here I

deliberately did not directly trust the `distance_matrix` attached to the CSV, but instead recomputed a Euclidean distance matrix from the coordinates myself, requiring the maximum difference between my own version and the CSV version to be below `1e-6` before passing (in practice this check passes, with a maximum difference of 0). The reason for this extra step is that the distances in the CSV were serialized into strings and read back, so they may carry floating-point truncation error. Rather than letting such error quietly seep into every fitness computation, I would rather use the clean version recomputed from the coordinates as the single source of distance. The distance matrix that GA and PSO subsequently see is all this self-computed version.

2.2 Genetic Algorithm (GA)

GA here uses permutation encoding directly. An individual is a permutation of `0` to `n-1`, representing the order in which cities are visited. The greatest benefit of this encoding is that as long as every operator preserves "permutation in, permutation out," the entire search process needs no repair step at all. The operator settings are as follows.

- Selection: tournament, $k = 3$
- Crossover: order crossover (OX), rate $= 0.8$
- Mutation: swap, one fixed swap per non-elite offspring
- Elitism: keep the 2 best individuals each generation

I chose OX rather than the numeric crossover used in the previous two assignments because a permutation cannot be blended dimension by dimension the way real numbers can, otherwise it would produce duplicated or missing cities. OX keeps a contiguous segment from one parent and fills the remaining positions according to the relative order of the other parent, inherently guaranteeing that the offspring is still a valid permutation.

```python
# lib/ga.py
def _order_crossover(p1, p2, rng):
    """OX: keep a random segment of one parent, fill the rest from the other."""
    a, b = np.sort(rng.choice(len(p1), size=2, replace=False))
    b += 1  # half-open segment [a, b)
    return _ox_child(p1, p2, a, b), _ox_child(p2, p1, a, b)


def _ox_child(donor, filler, a, b):
    n = len(donor)
    child = np.full(n, -1, dtype=donor.dtype)
    child[a:b] = donor[a:b]
    taken = set(donor[a:b].tolist())

    fill = [city for city in np.roll(filler, -b) if city not in taken]
    positions = [(b + k) % n for k in range(n - (b - a))]
    for pos, city in zip(positions, fill):
        child[pos] = city
    return child
```

Mutation uses the simplest swap, randomly picking two positions to exchange, which likewise does not break the permutation structure. Worth noting is that this swap is not "happening with some probability" but is done exactly once for every non-elite offspring, amounting to a stable amount of perturbation given to the whole population. Elitism keeps the 2 best individuals of the current generation, ensuring that the hard-won short routes are not altered by the next generation's crossover or mutation, so that convergence does not regress. The stopping condition is simply running a fixed 500 generations, with no additional early-stopping criterion.

2.3 Particle Swarm Optimization (PSO)

Compared with GA, the troublesome point of PSO is that it was originally designed for continuous space, whereas the solution to TSP is a discrete permutation. In the end I decided to reuse the random-keys idea from the previous program, letting each particle's position be a continuous vector (initialized in $[0, 1]^n$), and only converting this vector into a permutation with `argsort` when evaluating fitness, that is, sorting the continuous keys from small to large so that the resulting index order is the visiting order. This way the velocity update can stay entirely in continuous space and apply the standard PSO formula, with no need to devise separate rules for discrete space.

```python
# lib/pso.py: pso_minimize()
def evaluate(position: np.ndarray) -> float:
    return fitness_fn(np.argsort(position))

# ... swarm / pbest / gbest initialization omitted
for t in range(1, max_iter + 1):
    w = w_max - (w_max - w_min) * (t - 1) / (max_iter - 1) if max_iter > 1 else w_min
    for i in range(pop_size):
        r1 = rng.random(n)
        r2 = rng.random(n)
        vel[i] = (
            w * vel[i]
            + c1 * r1 * (pbest_pos[i] - pos[i])
            + c2 * r2 * (gbest_pos - pos[i])
        )
        vel[i] = np.clip(vel[i], -v_clamp, v_clamp)
        pos[i] = pos[i] + vel[i]
        # ... evaluate + pbest / gbest update omitted
```

The inertia weight ω decreases linearly from 0.9 to 0.4, keeping more of the old velocity early on to favor exploration and converging later. Here I use $(t-1)/(\text{max\_iter}-1)$ rather than $t/\text{max\_iter}$ so that the last generation lands exactly at $\omega_{\text{min}} = 0.4$. The cognitive and social coefficients are set to $c_1 = c_2 = 1.5$, and the velocity is clipped to $[-0.5, 0.5]$. What is somewhat special is that the position itself undergoes no clipping or repair, because `argsort` only cares about the relative magnitude between the keys. A particle position drifting outside $[0, 1]$ does not affect the permutation it maps to, and forcibly pulling it back into range would instead disturb the search. The stopping condition is likewise a fixed 500 generations.

2.4 Fitness Function

The fitness of *Part I* is simply the pure roundtrip length, that is, the total distance of the entire closed route (including the edge returning from the last city back to the start), with the goal of making it as small as possible.

Because *Part II* adds the constraint that even-numbered cities cannot be connected, the fitness is changed to add a penalty on top of the roundtrip length for every forbidden edge used:

$$f_{\text{II}}(\text{tour}) = \text{roundtrip}(\text{tour}) + \lambda \cdot (\text{number of forbidden edges used})$$

where $\lambda = 2n \max(D)$, which for this 30-city subset works out to 6611.81. I deliberately set λ this large, because the penalty from a single forbidden edge ($\geq \lambda$) is enough to exceed the total length of any complete feasible route (feasible solutions for this subset fall around 1200), so as long as a feasible solution exists, a penalized infeasible solution is always worse than it. This effectively uses a "soft" penalty

to achieve the effect of "hard" exclusion. The search is naturally pushed toward the feasible region, yet without having to forcibly block at the operator level, which preserves implementation simplicity.

In implementation I keep the distance matrix `D` as is, without rewriting it, and instead maintain a separate boolean mask `forbidden_mask` marking which (i, j) are even–even edges (judged in 1-indexed terms). This way the geometric length of a feasible route is always the true one, and the penalty is merely an extra term added on top, so the two are less likely to be confused when compared separately.

3. Parameter Settings

The complete parameters of the two algorithms are organized in the table below.

Parameter	GA	PSO
Population / swarm size	100	50
Max iterations	500	500
Number of runs (seeds)	30	30
Selection	Tournament ($k = 3$)	—
Crossover	OX (rate $= 0.8$)	—
Mutation	Swap (once per offspring)	—
Elitism	2	—
Encoding	Permutation	Random-keys $[0, 1]^n \rightarrow$ `argsort`
Inertia weight ω	—	$0.9 \rightarrow 0.4$ (linear)
Cognitive / social c_1, c_2	—	1.5 / 1.5
Velocity clamp	—	$[-0.5, 0.5]$
Velocity init	—	uniform $[-0.1, 0.1]$
Fitness	Part I pure roundtrip. Part II adds $\lambda\cdot$ forbidden edges, $\lambda = 2n \max(D)$	Same as left (shared)

There is one point I should make clear first. GA's population size is 100 and PSO's swarm is 50, so the two are not equal. This difference comes from my reusing each one's default value from the previous programs rather than tuning it deliberately for this assignment. Its direct consequence is that GA performs 100 fitness evaluations per generation while PSO performs only 50, so when comparing computation time later one should remember that GA being slower is partly simply because it computes twice as many evaluations per generation, not entirely because a single evaluation is more expensive.

The experiment environment runs on CPU with Python 3.12 and NumPy. Timing uses `time.perf_counter()` wrapping the algorithm body (including the initialization of the population / swarm, but not data loading), and each cell reports the average time over 30 runs. The random seeds are fixed at 0 to 29, and GA and PSO run the same seed numbers to make the experiment reproducible. However, this is only consistency at the seed level. The initial solutions are not identical, because GA initializes permutations while PSO initializes continuous vectors, which simply are not in the same space.

4. Part I

4.1 Protocol

Part I runs GA and PSO on each of the three subsets of the first 12, 30, and 52 cities, for a total of 6 cells, each cell executed 30 independent times with seeds 0–29, 500 generations each. The representative run of each cell is taken as the one with the lowest fitness, and Best Distance and Best fitness both come from this same route, avoiding the two metrics coming from different runs and not matching. Note that the fitness definition of *Part I* is the pure roundtrip distance, so the Best fitness value the spec requires here equals Best Distance, the two columns having the same value. Average Distance is the distance average over the 30 runs, and Computation Time is the average wall-clock time per run.

4.2 Results

The table below gives the four metrics for the 6 cells. Best Distance and Best Fitness have the same value, as noted above.

Case	Algo	Best Distance	Best Fitness	Avg Distance	Comp Time (s)
12 cities	GA	209.47	209.47	209.47	0.753
12 cities	PSO	209.47	209.47	227.14	0.208
30 cities	GA	444.86	444.86	522.80	0.796
30 cities	PSO	494.12	494.12	604.80	0.218
52 cities	GA	850.38	850.38	943.79	0.881
52 cities	PSO	1019.25	1019.25	1237.96	0.233

4.3 Observations

On the smallest 12 cities, both GA and PSO found the same shortest value 209.47, which can reasonably be taken as the global optimum of this subset. The difference lies in stability. All 30 runs of GA converged to 209.47, while only 15 of PSO's reached the target, the rest landing on slightly longer routes, so its Avg Distance (227.14) is pulled up. This shows that when the search space is still small, both can find the optimal solution, but GA finds it more reliably.

Once the city count grows, GA's advantage shows up in both the Best and Avg columns at the same time. At 30 cities GA's best 444.86 is clearly shorter than PSO's 494.12, and at 52 cities it widens further to 850.38 versus 1019.25. The gap in average distance is even larger. At 52 cities GA averages 943.79 while PSO averages 1237.96. My understanding is that GA's OX and swap operate directly on the permutation structure, able to preserve and recombine good sub-route fragments. PSO controls the permutation indirectly through continuous keys, and as cities increase, a small perturbation of the keys can make the order produced by `argsort` jump substantially, making it harder to stably accumulate local structure.

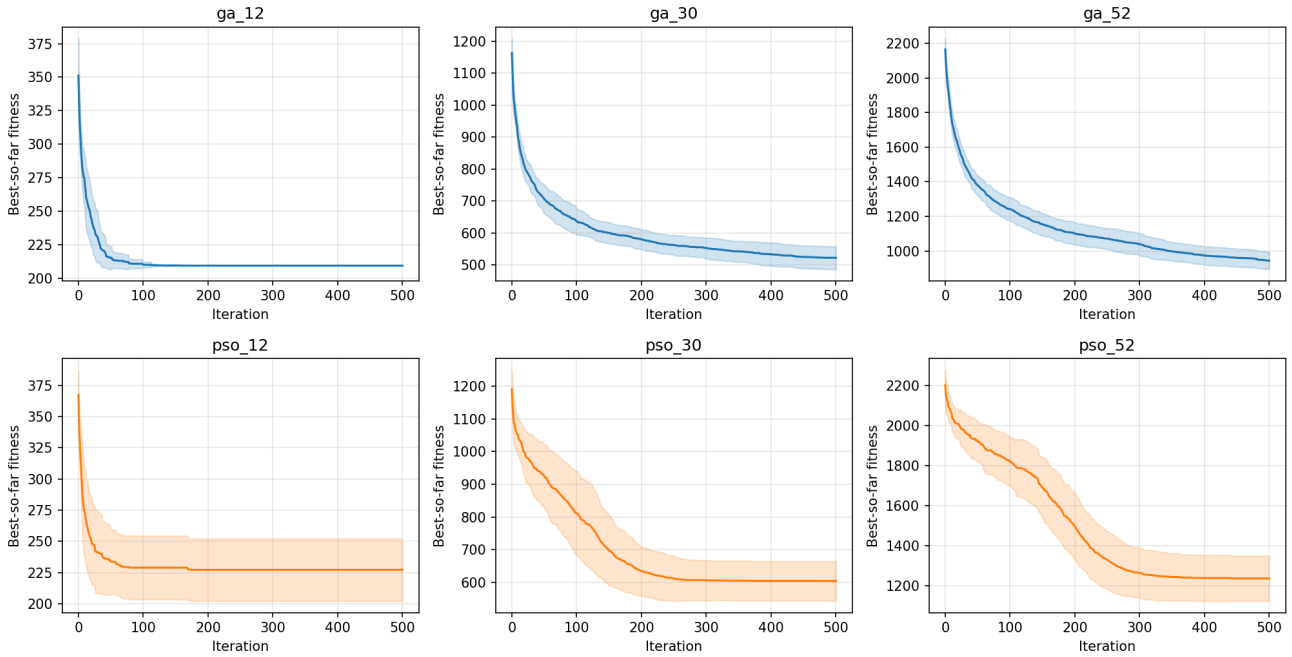


Figure 1. *Part I* convergence curves for the six cells. Each curve is the average best-so-far fitness over 30 runs, and the shaded band is ± 1 standard deviation.

The convergence curves also echo the observation above. In all three subsets GA's curve sits below PSO's, and the standard-deviation band almost collapses to a single line at 12 cities, corresponding to its 30/30 full convergence. The standard-deviation bands at 30 and 52 cities clearly widen, reflecting the larger spread of results across different seeds. This is visible for both GA and PSO, only that GA's overall level is lower.

As for Computation Time, PSO is about 3.5 to 4 times faster than GA on every subset. However, as noted in Section 3, GA's population is 100 while PSO's is only 50, so GA performs twice as many fitness evaluations per generation, which means a considerable part of this time difference is caused by population size and cannot be simply read as PSO's single iteration being cheaper. Taking this into account, the fairer conclusion is that GA trades more computation for a shorter and more stable route.

5. Part II

5.1 Protocol

Part II takes only the first 30 cities and adds the constraint that even-numbered cities cannot be connected to each other. In 1-indexed terms, among the first 30 cities there are 15 even-numbered cities, which together form 105 forbidden edges pairwise. The fitness switches to the penalized form of Section 2.4, with $\lambda = 6611.81$. Again GA and PSO each run 30 seeds, 500 generations each, with the representative run taken as the one with the lowest fitness.

Besides the four basic metrics, *Part II* additionally reports three numbers related to feasibility, namely the number of forbidden edges used by the representative route `forbidden_edge_count`,

the shortest pure distance among all runs `min_pure_distance` (whether feasible or not), and an `infeasibility_flag` marking whether the representative route still steps on a forbidden edge. I deliberately do no repair or retry. Once an infeasible case appears it is faithfully marked and reported as usual, rather than being hidden away.

5.2 Results

Algo	Best Distance	Best Fitness	Avg Distance	Comp Time (s)	Forbidden Edges	Min Pure Distance	Infeasible
GA	1190.74	1190.74	1216.89	0.881	0	1190.74	No
PSO	1372.62	1372.62	1400.70	0.250	0	1201.77	No

Both algorithms' representative routes are feasible (zero forbidden edges), so Best Fitness has no penalty added and still equals Best Distance. Special attention should go to the PSO row. Its `min_pure_distance` (1201.77) is shorter than its own Best Distance (1372.62), which is not a contradiction but is because this shortest pure distance comes from a route that, although shorter, actually steps on a forbidden edge. It is geometrically short but illegal, so it was not chosen as the representative.

5.3 Observations

Both algorithms' representative routes are feasible, indicating that the very large λ indeed achieved its intended effect. As long as a feasible solution exists in the population, its fitness must dominate all infeasible solutions, and the representative run naturally lands on the shortest feasible route.

However, spreading out the 30 runs, the reliability of GA and PSO differs greatly. All 30 of GA's runs found a feasible route, with not a single one ending on a forbidden edge. PSO, on the other hand, had 15 runs, a full half, whose final gbest still carried a forbidden edge. Since the penalty is as high as 6611, a route with a forbidden edge has its fitness shoot straight above 7800, which means those 15 runs did not "find a worse feasible solution" but failed from start to finish to assemble any fully legal route. This is consistent with the trend seen in *Part I*. PSO's random-keys encoding is clearly strained on this kind of problem with hard adjacency constraints.

Here too the Avg Distance column needs to be made clear, to avoid misreading. It is the pure-distance average over the 30 runs, and it averages all of them together regardless of feasibility. For PSO, this 1400.70 mixes in the pure distances of those infeasible runs, so it is not the "average length of feasible solutions." Likewise PSO's `min_pure_distance` (1201.77) comes from an infeasible route. In other words, the PSO numbers in *Part II* must be read together with the Infeasible column. Looking only at distance would underestimate the degree to which it violates the constraint. For GA, because all runs are feasible, this problem does not exist, and its average 1216.89 is a genuine feasible-solution average.

In terms of route quality itself, GA's feasible shortest route (1190.74) is also shorter than PSO's (1372.62), so in *Part II*, whether judged by feasibility rate or route length, GA is the better one.

6. Discussion (ii)

This section corresponds to the spec's Discussion (ii), plotting the best roundtrip routes of both Parts. The city labels in all figures are 1-indexed, consistent with the earlier narrative.

The best routes of *Part I*'s three subsets are plotted per algorithm below, each panel showing the 12-, 30-, and 52-city subsets from left to right.

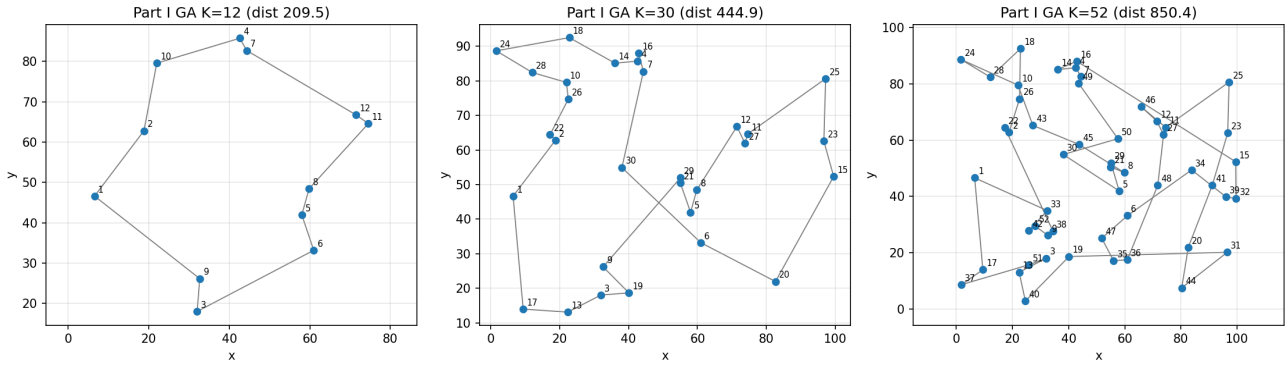


Figure 2. *Part I* best routes, GA, for the 12-, 30-, and 52-city subsets.

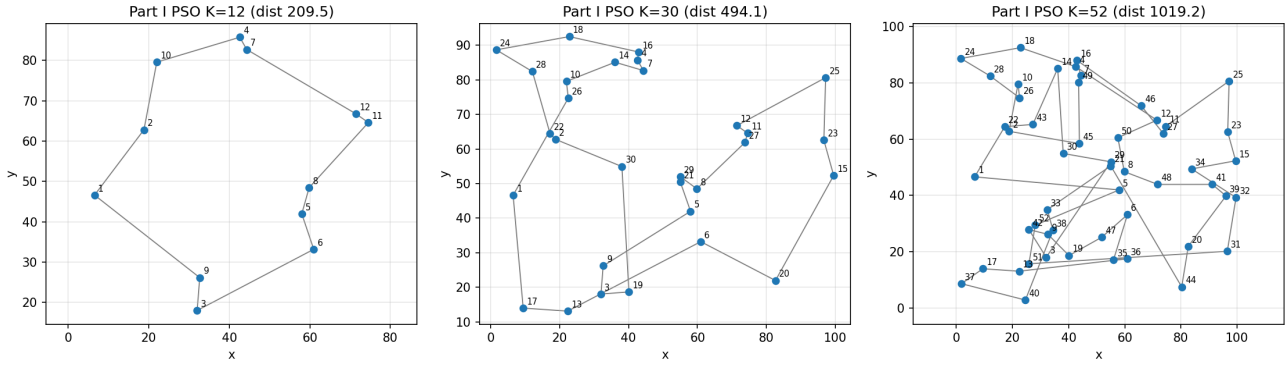
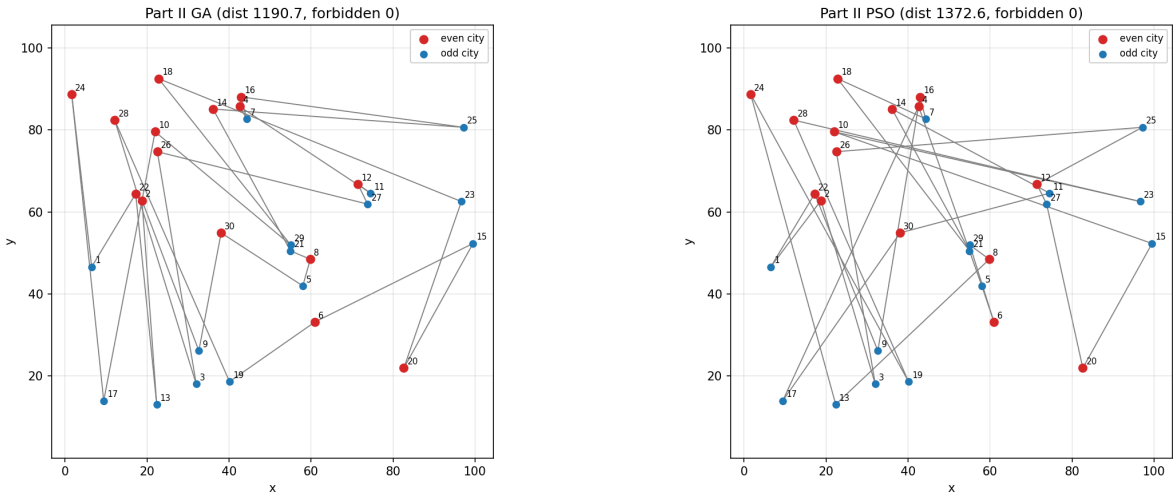


Figure 3. *Part I* best routes, PSO, for the 12-, 30-, and 52-city subsets.

The two routes for 12 cities are almost identical, both settling into the same crossing-free loop, consistent with both reaching 209.47. After the city count grows, GA's 30- and 52-city routes look more "converged" than PSO's, with fewer crossings and detour edges, which is consistent with GA's shorter routes in the table. PSO's routes still show several long edges that visibly stretch the loop open.

The two best routes of *Part II* are as follows. The figures also mark even and odd cities in different colors, and any forbidden even–even edge used would be shown as a red dashed line.



(a) GA (b) PSO

Figure 4. *Part II* best routes: (a) GA (length 1190.74, 0 forbidden edges), (b) PSO (length 1372.62, 0 forbidden edges).

Neither figure shows any red dashed line, directly confirming the feasibility of the representative routes. The entire loop successfully avoids even cities being connected to each other at adjacent positions. By comparison one can also see that, in order to route around these forbidden adjacencies, the route takes a few more turns in regions where even cities are dense, which is exactly the cost the constraint brings.

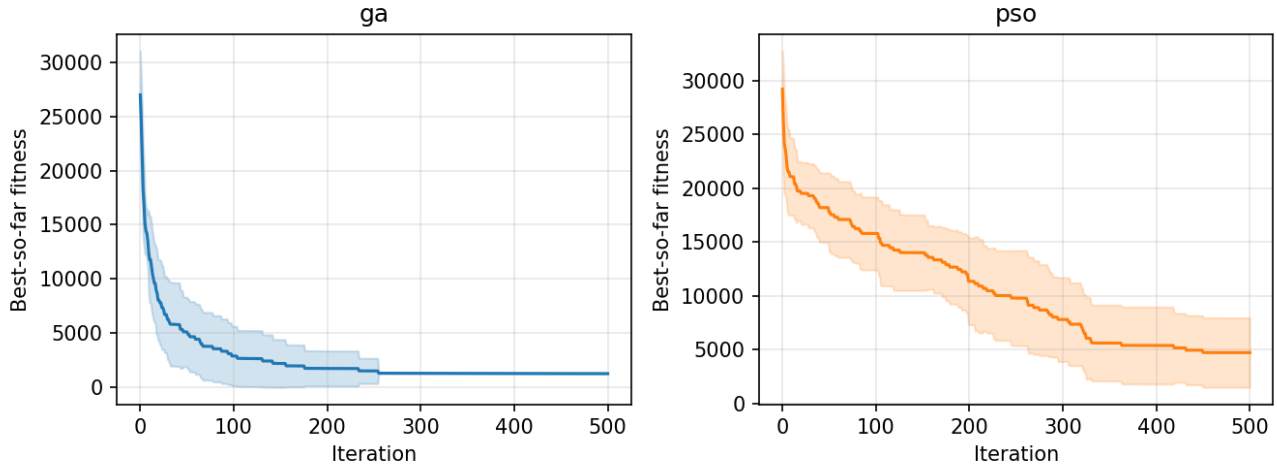


Figure 5. *Part II* convergence curves. Average best-so-far fitness over 30 runs each for GA and PSO, with the shaded band being ± 1 standard deviation.

Part II's convergence curves supplement a phenomenon mentioned earlier. GA's curve descends smoothly to the feasible-solution level with a very narrow standard-deviation band. PSO's standard-deviation band stays wide throughout, precisely because those 15 runs perpetually stuck in the infeasible region stretch it open.

7. Conclusion

Moving this same pair of GA and PSO onto TSP, the conclusions I obtained are actually quite consistent. On this problem, GA is comprehensively more stable than PSO. Across *Part I*'s three scales, GA's best and average routes are both shorter, and the gap grows the more cities there are. After *Part II* adds the forbidden-edge constraint, GA not only has shorter routes but also has all 30 runs feasible, while PSO has half of its runs unable to assemble even one legal route. PSO looks several times faster in wall-clock time, but a large part of that is to be attributed to its swarm being only half the size of GA's population, computing half as many fitness evaluations per generation, so this "fast" cannot be taken directly as an efficiency advantage.

I believe the root of the gap lies in the solution representation. GA's permutation encoding paired with OX and swap has all operations land directly on the permutation structure, needing no repair while preserving good sub-routes. PSO's random-keys controls the permutation indirectly through continuous keys, and when cities grow many and a hard adjacency constraint is encountered, this layer of indirection becomes a burden. So if I had to pick one for this kind of constrained combinatorial optimization problem, I would choose GA. PSO is not unusable, but it is natively designed for continuous space, and applying it to discrete permutations is ultimately taking a detour.

References

[1] ZIYA. *Traveling Salesman Problem (TSPLIB Dataset)*. Kaggle. https://www.kaggle.com/datasets/ziya07/traveling-salesman-problem-tsplib-dataset